

KINEMATIC MIRROR MOUNT ■ USB & WI-FI CONTROL

Command Reference Manual

Every command the mount accepts, over either transport — plus first-time setup, reconnecting over Wi-Fi, and the `mount` command-line client.

1 OVERVIEW

The mount has two independent axes, each a brushed DC motor with an analog Hall position sensor:

- **Axis 1** — motor M1, sensor a0
- **Axis 2** — motor M2, sensor a1

Commands are plain text, one per line, and are identical over both transports. In the syntax below, `N` is the axis number (1 or 2) and sensor values are raw ADC counts (0–4095).

Transport	Details
USB serial	115200 baud, newline line endings. Always available; the only transport that accepts provisioning commands.
WiFi (TCP)	Port 3333, discovered via mDNS as <code><name>.local</code> , where <code><name></code> is the unit's name reported by status (unnamed units fall back to <code>mirrormount-<id></code>). Requires authentication (Section 5).

Responses are text lines, mirrored to USB *and* the connected WiFi client.

2 GETTING THE MOUNT CLIENT

Everything in this manual is driven by the `mount` command-line client, which lives in the companion repository `physiverse_monolithic_motorized_mirror_mount_client` (Python, macOS/Linux). **Start by cloning and installing it** — no `mount` command in this manual will run until you do:

```
git clone https://github.com/physiverse/\
physiverse_monolithic_motorized_mirror_mount_client.git
cd physiverse_monolithic_motorized_mirror_mount_client
pip install .                # provides the "mount" command
python3 -m mount_client --help # or run it without installing
```

Command	Description
<code>provision -ssid <ssid> -wifi-password <pw> [-device-password <pw>] connect [<host> <password> [tcp_port]]</code>	First-time setup over USB (Section 3): saves WiFi credentials and, if given, the command password, then waits for the mount to join the network and prints its .local name. Interactive session over WiFi (authenticated). With <code>-usb</code> instead of <code>host/password</code> : serial session, no password needed. Ctrl-C or Ctrl-D quits; on USB every keystroke is forwarded as typed.
<code>status [<host> <password>]</code>	One-shot report of the unit's name and both axes, plus WiFi state, over either transport (same transport selection as <code>connect</code>).
<code>factory-reset</code>	Clear WiFi + command password over USB; axis calibration is kept.

Transport selection, shared by `connect` and `status`: give `<host> <password>` for WiFi (TCP port defaults to 3333; `<host>` may be a .local name or an IP), or pass `-usb [PORT]` for serial — with no PORT the board is auto-detected (`/dev/cu.usbmodem*` on macOS); `-port PORT` is a synonym. `-baud` overrides the serial rate (default 115200).

```
mount status mirrormount-ac88.local mountpass # one-shot over WiFi
mount connect mirrormount-ac88.local mountpass # interactive over WiFi
mount connect --usb                          # interactive over serial
mount factory-reset                          # over USB only
```

Connecting over USB

USB is the transport that always works — before the mount has ever seen a network, and whenever you need it back. With the client installed, connect the board to your computer with a USB-C cable and run:

```
mount connect --usb # interactive session: type commands, Ctrl-C to quit
mount status --usb  # one-shot report of name, axes and wifi, then exits
```

No password is needed over USB — authentication applies to WiFi clients only. The serial port is auto-detected (`/dev/cu.usbmodem*` on macOS); pass `-usb PORT` if more than one board is plugged in. Any serial console works just as well at 115200 baud (for example `pio device monitor` or `screen /dev/cu.usbmodem3101 115200`) — the commands are the same, you just type them yourself.

If the mount is new, continue with first-time setup below; if it is already provisioned, Section 4 covers reaching it over WiFi.

3 FIRST-TIME SETUP (OVER USB)

With the client installed (Section 2), the whole procedure is one command:

```
mount provision --ssid MyNetwork --wifi-password wifipass \  
--device-password mountpass
```

It auto-detects the serial port (`-port` overrides), stores the credentials on the device, waits until the mount joins the network, and prints the `connect` line for future use.

Manually, connect the mount by USB and open a serial console (e.g. `pio device monitor`). Then:

1. Set the device login password:
`setpass <password>`

2. Give it your WiFi network:

```
wifi <ssid> <password>
```

3. It confirms and prints its address, e.g.

```
wifi: CONNECTED ip=192.168.1.42 name=mirrormount-ab12.local
```

Both are stored in flash and reconnect automatically on power-up — you should not need USB again for day-to-day use (Section 4). There are **two separate secrets**: the *login password* (setpass, authenticates clients) and the *WiFi credentials* (wifi, joins your network).

Command	Description
setpass <password>	Set the login password (stored hashed). USB only.
wifi <ssid> <password>	Save WiFi credentials and connect. USB only. Password may contain spaces.
wifi	Show connectivity status (SSID, IP, name, signal).
factory-reset	Clear WiFi + password (keeps axis calibration). USB only.

4 CONNECTING OVER WIFI LATER

Setup (Section 3) is done *once*. The credentials live in flash, so on every power-up the mount rejoins the same network by itself and starts listening on TCP port 3333. From then on a session is a single command — no USB, no re-provisioning:

```
mount connect mirrormount-ac88.local mountpass
mount status mirrormount-ac88.local mountpass # one-shot, then exits
```

The two arguments are the mount's address and the *login* password you chose with setpass (or -device-password) — not your WiFi password. Add a third argument only if the firmware was built to listen on a non-default TCP port: `mount connect mirrormount-ac88.local mountpass 4444`.

Finding the address again

Prefer the .local name: it is the unit's own name, is printed at provisioning time, and never changes. The IP address comes from your router's DHCP lease and *can* change between power cycles, so treat it as a fallback rather than the thing you write down.

If you no longer know either one, ask the mount. Plug in USB and run:

```
mount status --usb
```

The report opens with the unit's name. A mount that answers `mount: planksprank` is reachable at `planksprank.local`. The trailing `wifi` line then gives the current SSID, IP, mDNS name and signal strength, e.g. `wifi: CONNECTED ip=192.168.1.42 name=planksprank.local`. The name is reported either way, so it is still there to read when the mount is *not* on the network; the SSID and IP appear only once it is connected. The same two lines come from typing `status` and `wifi` in a serial console, or in an already-open WiFi session. Failing that, look for the mount's name in your router's client list.

When you do need USB again

Only for things that change the stored secrets or the network itself: re-run `mount provision` (or the `wifi` command) after moving the mount to a different network or changing that network's password; run `setpass` to change the login password; run `factory-reset` to clear both. Everything else — motion, calibration read-back, status — works over WiFi.

There is no password recovery: the login password is stored hashed, so a forgotten one is replaced over USB with `setpass <new>`, not read back.

Troubleshooting

Symptom	Likely cause and fix
cannot connect ... Name or service not known	The <code>.local</code> name did not resolve. mDNS does not cross subnets and some corporate/guest networks block it — connect from the same LAN, or use the IP address reported by <code>mount status -usb</code> .
cannot connect ... Connection refused, or a timeout	The mount is powered but not on the network (or is on a different one). Check with <code>mount status -usb</code> ; if it reports anything other than <code>wifi: CONNECTED</code> , re-provision.
authentication failed (wrong password?)	Wrong login password, or none was ever set. Set one over USB with <code>setpass <password></code> .
Connects, then drops mid-session	Weak signal or the mount lost power. In-flight <code>seekN/home</code> moves still finish on their own; an open-ended manual drive is auto-stopped (Section 7).

5 WIFI AUTHENTICATION

On each TCP connection the device issues a challenge; the client proves it knows the password without sending it:

```
device -> NONCE <hex>                (random, per connection)
client -> AUTH <hex>    hex = HMAC-SHA256(K, nonce)
device -> OK                (authenticated) -- or -- DENIED
```

where $K = \text{SHA256}(\text{password})$. After OK, the same commands as the serial console are accepted. The mount client (Section 2) performs this handshake for you.

6 CONTROL COMMANDS

Accepted on both transports (WiFi requires authentication first).

Command	Description
<code>seekN <target> <speed> [stall_ms]</code>	Closed-loop move until sensor = target. speed is a magnitude; direction is automatic. Clamped to the calibrated range; gives up after <code>stall_ms</code> of no progress (default 2500).
<code>homeN [speed]</code>	Drive axis N to its home position (saved home, else range midpoint).
<code>home</code>	Home <i>both</i> axes simultaneously.
<code>tickN <$\pm n$> [spd] [ms]</code>	Fine adjustment: one continuous drive worth <code>n</code> minimal ticks (sign of <code>n</code> = direction). Pulse size defaults to the per-axis saved values (speed 100 / 5 ms unless changed).
<code>s</code>	Stop both motors.
<code>r [n]</code>	Read <code>a0/a1</code> , averaged over <code>n</code> samples (default 1).
<code>status</code>	Report the unit's name (<code>mount: <name></code>), then both axes: speed, polarity, range, home.
<code>?</code>	Help.

Calibration and low-level motor commands exist in the firmware but are intentionally not documented here — see the firmware repo's `PROTOCOL.md` if you know you need them (USB recommended).

7 NOTES

- **Calibration** (polarity, range, home) is per-axis, stored in flash, and survives reboot and factory-reset.
- `seekN/home` moves are self-terminating (range clamp + stall guard). If the WiFi client disconnects mid-move, they finish.
- Open-ended manual drives run until `s`; if the authenticated WiFi client is lost, this is the one motion that is auto-stopped (fail-safe).
- The sensor is non-linear near its travel extremes (“soft shoulders”); the calibrated range excludes them, and seek targets are clamped into it.

8 TYPICAL SESSION

```
# over USB, once:
setpass labpass123
wifi MyNetwork mypassword

# then, every session after that, over WiFi (or over USB):
mount connect mirrormount-ac88.local labpass123 ; or: mount connect --usb
status
home
seek1 2300 150
seek2 2100 150
tick1 -5          ; small nudge, axis 1, toward lower sensor values
r 16
```